

TOWARDS REVERSIBLE MODEL MERGING FOR LOW-RANK WEIGHTS

Anonymous authors

Paper under double-blind review

ABSTRACT

Model merging seeks to map multiple fine-tuned parameterizations to a consolidated weight representation that maintains high utility on all source tasks. Existing merging methods can approximate, but generally do not recover, the per-task performance of the corresponding individually fine-tuned models, and they have rarely been analyzed when the task updates are stored as low-rank factors, either through low-rank adaptation (LoRA) or post-training singular value decomposition (SVD). We show that directly applying standard merging operators to low-rank weights induces substantial degradation in the resulting merged model. In response, we formulate an alternative objective: rather than collapsing all adapters into a single parameter vector, we learn a compact basis (e.g., with storage comparable to two or more models) from which each task-specific model can be reconstructed by a linear combination. This formulation treats merging as the construction of a reconstruction-capable model space rather than as the production of one aggregate model. It therefore permits controlled “reversion” to an individual task model at inference time, reflecting the empirical fact that a single merged model cannot be assumed to dominate a specialist on its own task. We instantiate this principle as Reversible Model Merging (RMM), a data-free and flexible procedure with a closed-form solution for the optimal basis directions and task-specific reconstruction coefficients. Experiments over multiple datasets, compression regimes, and model scales show that RMM consistently outperforms existing merging baselines and preserves the accuracy of low-rank compressed models by a substantial margin.

1 INTRODUCTION

Parameter-efficient fine-tuning methods, including Low-Rank Adapters (LoRA) (Hu et al., 2021), provide a standard mechanism for adapting large language models to heterogeneous downstream tasks while avoiding full-parameter retraining. Orthogonally, compression methods such as post-training singular value decomposition (SVD) truncation reduce the storage and inference footprint of fine-tuned updates, thereby introducing an additional efficiency axis (Ryu et al., 2023; Ping et al., 2024; Wang et al., 2025b;a). Model merging has developed in parallel as a mechanism for consolidating several task-specific parameterizations into one deployable model. Recent methods, from direct parameter interpolation to optimization-based merging, show that a merged model can attain competitive aggregate performance across source tasks without access to the original training data (Matena & Raffel, 2022; Yang et al., 2024; Yadav et al., 2023; Wei et al., 2025).

A central limitation remains: even strong merging algorithms typically do not match the validation performance of the original task-specialized models. The discrepancy becomes especially severe when the objects being merged are low-rank representations produced by LoRA fine-tuning or post-training SVD truncation. Although conventional merging yields a single model that can be evaluated on multiple tasks, it also projects distinct task-specific updates into one shared parameterization. This projection can induce cross-task interference and can remove the specialized structure learned by each fine-tuned model (Gargiulo et al., 2025).

Low-rank adapters and SVD-truncated updates are individually compact, but large collections of such objects create a separate scalability constraint. In multi-task, federated, and continual-learning deployments, systems may accumulate dozens or hundreds of task-specific low-rank adapters, each

of which must be stored, indexed, and selected during deployment. Maintaining the full collection reduces the aggregate storage benefit of low-rank parameterization and complicates model management. This setting motivates merging procedures that operate directly on low-rank factors, with the dual goal of reducing redundant storage and supporting efficient sharing, deployment, and transfer across tasks.

We therefore first characterize how existing merging operators behave on low-rank compressed models. The analysis is expressed in terms of deltas, i.e., the weight differences between each fine-tuned model and the common pre-trained initialization. Standard merging procedures combine these deltas into a single parameter update under an implicit assumption of linear compatibility. Our experiments show that applying this operation directly to low-rank deltas produces catastrophic degradation. The failure is attributable to two interacting mechanisms: the restricted expressivity of low-rank updates and the misalignment of independently optimized task-specific subspaces, which generates severe interference after combination.

This observation suggests a different formal objective. Instead of requiring the merging procedure to output exactly one final model, we ask whether a small number of retained components can parameterize accurate reconstructions of the original compressed models. Under this relaxation, the merged object is a compact set of weights that functions as a shared basis, and each task-specific model is recovered by an associated linear coefficient vector. The resulting design space preserves complementary representational directions across a limited set of basis models and enables post-merging “reversion” to the corresponding adapter. Thus, the merging target is no longer a single aggregate parameterization but a reconstruction-capable model space.

On this reconstruction-oriented basis, we derive a framework that specifies the optimal basis for minimizing squared reconstruction error. The resulting method, Reversible Model Merging (RMM), casts merging as a basis-selection problem over task vectors: the objective is to identify a compact set of directions and a coefficient matrix from which the original low-rank task updates can be reconstructed. RMM admits a closed-form solution that jointly determines the basis and the task-specific linear coefficients. Empirically, RMM is evaluated over multiple datasets and model configurations and consistently exceeds conventional merging baselines by a large margin. The method retains the storage advantages of low-rank parameterization while exposing an explicit hyperparameter that controls the storage–performance trade-off through the number of retained reversible components.

2 RELATED WORK

Model merging denotes a family of procedures for consolidating information from multiple fine-tuned models into a smaller set of deployable weights. Existing methods can be organized into two broad classes: data-free manipulations in parameter space and optimization-based procedures that use data, auxiliary objectives, or estimated parameter importance. We review representative methods from both classes.

Task Arithmetic: Task arithmetic (Ilharco et al., 2023) defines a task delta as the parameter difference between a fine-tuned model and its pre-trained initialization. The method treats these deltas as compositional vectors in weight space: adding a delta to the base model introduces the associated capability, subtracting it can attenuate an undesired behavior, and scaled combinations can combine multiple capabilities. This linear formulation provides a simple data-free mechanism for composing model behavior directly in parameter space.

DARE: Direct delta addition can fail when independently fine-tuned models contain conflicting or redundant parameter updates. DARE (Drop and Rescale) (Yu et al., 2024) addresses this issue through randomized pruning of task deltas. The procedure drops a large fraction of parameter changes—up to 90-99%—under the hypothesis that many updates are nonessential or interference-inducing, and then rescales the retained entries to preserve the magnitude of the update. The intended effect is to retain salient task-specific modifications while reducing cross-model interference.

TIES: TIES-merging (Yadav et al., 2023) replaces randomized pruning with a deterministic conflict-resolution procedure. It consists of Trim, Elect Sign, and Merge phases. The Trim step retains only high-magnitude entries, the Elect Sign step determines a consensus sign at each parameter position and removes sign-disagreeing contributions, and the Merge step averages the remaining sign-consistent updates. This procedure is designed to reduce destructive interference by enforcing

parameter-level agreement before aggregation. Motivated by a related interference-reduction objective, EMR-Merging (Elect, Mask and Rescale) elects a unified model and augments it with task-specific binary masks and rescalers that shift the unified weights toward each task (Huang et al., 2024).

Other Merging Methods: Beyond data-free parameter operations, several merging methods solve explicit optimization problems or require a small auxiliary dataset. Model soups (Wortsman et al., 2022) select or refine interpolation weights by minimizing validation loss, thereby biasing the merge toward configurations with better generalization. Fisher-weighted averaging (Matena & Raffel, 2022) uses second-order information to assign greater influence to parameters estimated to be important for task performance. Other methods formulate merging coefficients as optimization variables: RegMean (Jin et al., 2023) and AdaMerging (Yang et al., 2024) solve small-scale objectives based on prediction discrepancies between merged and individual models and prediction entropy on unlabeled data, respectively. Another recent formulation minimizes the loss difference between the merged model and the individual fine-tuned models and derives a tractable surrogate through a first-order Taylor expansion (Wei et al., 2025). These approaches can improve task performance, but they commonly require labeled or unlabeled data, additional optimization, or both, which restricts their use when merging must remain data-free and retraining-free.

3 PRELIMINARIES

We begin with the standard model-merging objective in which a collection of fine-tuned models is converted into a single unified parameterization. Let $\{\theta_1, \dots, \theta_n\}$ denote n models fine-tuned on n distinct tasks. A merging algorithm $\mathcal{M}$ maps this collection to merged weights $\theta_{\mathcal{M}} = \mathcal{M}(\{\theta_i\}_{i=1}^n)$. The corresponding objective is to choose a procedure $\mathcal{M}$ that maximizes mean validation performance over the task set:

$$\max_{\mathcal{M}} \frac{1}{n} \sum_{i=1}^n \text{Perf}(\theta_{\mathcal{M}}, \mathcal{D}_i), \quad (1)$$

where $\mathcal{D}_i$ is the held-out validation set for task i , and Perf denotes the task-appropriate metric, such as accuracy or F1 score. We assume that all fine-tuned models share the same pretrained initialization θ_{pre} . For layer l , the task update, or delta, of model i relative to the base weights is $\Delta_i^l = \theta_i^l - \theta_{\text{pre}}^l$, with $\Delta_i^l \in \mathbb{R}^{m \times d}$ representing the weight difference for an $m \times d$ layer.

3.1 LOW-RANK COMPRESSED MODELS

Fine-tuned updates are often represented in low-rank form to reduce storage, training, and inference costs. We consider two common mechanisms for obtaining such representations:

LoRA fine-tuning: During fine-tuning, each adapted layer is augmented with trainable low-rank factors $\mathbf{A} \in \mathbb{R}^{m \times r}$ and $\mathbf{B} \in \mathbb{R}^{r \times d}$, where $r \ll \min(m, d)$. The pretrained weights are held fixed, and optimization is restricted to $\mathbf{A}$ and $\mathbf{B}$.

Post-training SVD truncation (PT-SVD): After full fine-tuning, the update Δ_i^l is factorized by SVD and truncated to its top- r singular components, producing the low-rank approximation $\hat{\Delta}_i^l = \mathbf{A}_i^l \mathbf{B}_i^l$, with $\mathbf{A}_i^l \in \mathbb{R}^{m \times r}$ and $\mathbf{B}_i^l \in \mathbb{R}^{r \times d}$ for $r \ll \min(m, d)$.

Under either mechanism, the layer- l update of model i is stored as $\hat{\Delta}_i^l = \mathbf{A}_i^l \mathbf{B}_i^l$, where $\mathbf{A}_i^l \in \mathbb{R}^{m \times r}$ and $\mathbf{B}_i^l \in \mathbb{R}^{r \times d}$, and r is the compression rank. The merging problem is therefore transformed into the problem of aggregating compressed task deltas. The desired merged model integrates $\hat{\Delta}_i = \{\hat{\Delta}_i^l\}_{l=1}^L$ over all tasks:

$$\hat{\theta}_{\mathcal{M}} = \theta_{\text{pre}} + \mathcal{M}(\{\hat{\Delta}_i\}_{i=1}^n), \quad \max_{\mathcal{M}} \frac{1}{n} \sum_{i=1}^n \text{Perf}(\hat{\theta}_{\mathcal{M}}, \mathcal{D}_i), \quad (2)$$

where L denotes the number of layers. The remainder of the paper studies this low-rank setting and first evaluates how conventional merging procedures behave when their inputs are compressed factors.

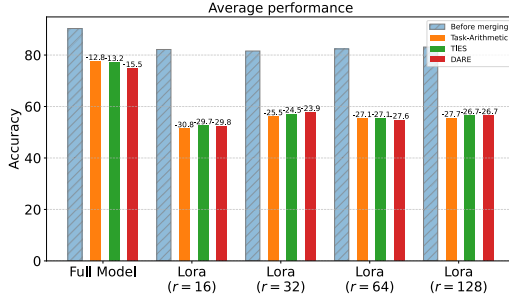


Figure 1: Average task performance for individual pre-merge models and for merged models produced by each merging method. The comparison includes fully fine-tuned models and LoRA fine-tuned models at ranks $r \in \{16, 32, 64, 128\}$.

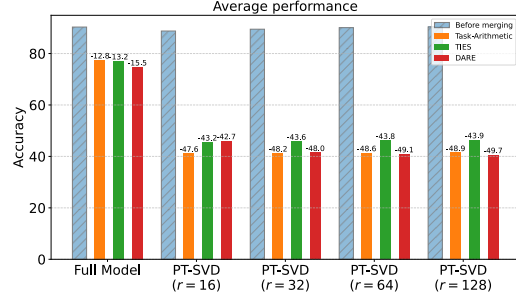


Figure 2: Average task performance for individual pre-merge models and for merged models produced by each merging method. The comparison includes fully fine-tuned models and PT-SVD compressed models at ranks $r \in \{16, 32, 64, 128\}$.

4 MOTIVATING OBSERVATION

We next evaluate conventional merging strategies when the merged objects are the low-rank factors defined above. For each layer l , prior work considers two direct extensions of standard merging to this setting (Tang et al., 2025; Zhao et al., 2025; Zheng et al., 2025; Zhang et al., 2025):

- **Combined merging:**

$$\hat{\theta}_{\mathcal{M}}^l = \theta_{\text{pre}}^l + \mathcal{M}(\{A_i^l B_i^l\}_{i=1}^n),$$

where each low-rank update is first materialized in full-rank form and the resulting matrices are then merged.

- **Separate merging:**

$$\hat{\theta}_{\mathcal{M}}^l = \theta_{\text{pre}}^l + \mathcal{M}(\{A_i^l\}_{i=1}^n) \mathcal{M}(\{B_i^l\}_{i=1}^n),$$

where the two low-rank factors are merged separately and then recomposed.

Both formulations are well defined, but the combined formulation is storage-inefficient. Materializing and storing the merged update in combined form requires an $m \times d$ matrix, whereas the separate formulation stores only low-rank factors $A_i^l \in \mathbb{R}^{m \times r}$ and $B_i^l \in \mathbb{R}^{r \times d}$. Because $r(m + d) \ll md$ when $r \ll \min(m, d)$, the combined strategy removes the storage advantage that motivates low-rank compression. For example, with $m = d = 1024$ and $r = 16$, the combined representation stores $md = 1,048,576$ parameters, while the separate representation stores $r(m + d) = 16 \times (1024 + 1024) = 32,768$ parameters.

This corresponds to a **32 $\times$ reduction in storage** for the factorized representation. We therefore treat separate merging as the practically relevant baseline for compressed models.

Using the separate formulation, we apply several existing merging baselines to low-rank compressed models under different compression ratios. RoBERTa-base (Zhuang et al., 2021) serves as the pre-trained model, and evaluation is performed on 5 GLUE tasks (Wang et al., 2018); additional details are provided in Appendix A.3. Figures 1 and 2 show that compressed-model merging produces a *catastrophic drop* in average task performance relative to full-rank merging. We identify two explanatory factors: (i) **Amplified task interference**: compression reduces representational capacity, making low-rank updates more sensitive to destructive interactions during aggregation; (ii) **Loss of coupling between A and B**: standard merging treats A_i^l and B_i^l as independent matrices even though the represented update is their product, which can introduce structural misalignment after recomposition. These factors jointly degrade merged low-rank models and indicate a structural mismatch between traditional merging operators and compressed representations.

The preceding result implies that, under conventional single-model merging, the task-specific performance of the original low-rank adapters is not reliably recoverable. We therefore consider a different question: *Can the original compressed models be reconstructed, or closely approximated, from a*

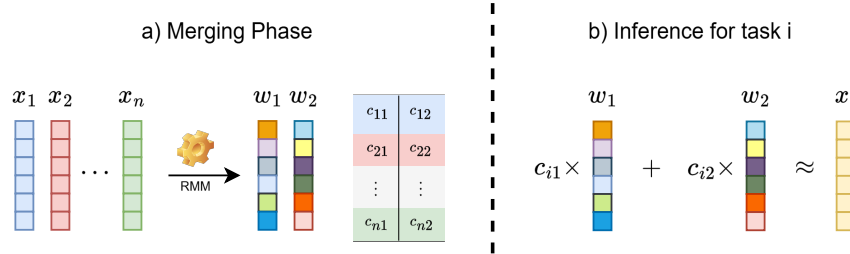


Figure 3: Schematic of the RMM framework. **Left:** The offline merging phase estimates a compact basis (two directions in the illustration) together with task-specific coefficients. **Right:** During inference, target model i is reconstructed by applying its coefficient vector to the stored basis directions.

compact set of basis models whose size is determined by the storage budget? Instead of forcing all updates into one representation, we define merging as the construction of a *reconstruction-capable model space*. The next section formalizes this objective and introduces *Reversible Model Merging* (RMM).

5 REVERSIBLE MODEL MERGING

The empirical degradation observed for compressed-model merging indicates a structural limitation of single-output merging: one aggregate parameterization is generally insufficient to preserve the task-specific behavior of all low-rank models. RMM therefore replaces the single-weight-output requirement with a reconstruction objective. The method stores a small set of shared components, interpreted as basis models, from which each original task model can be approximately reconstructed.

Instead of mapping all n task-specific models to a single merged solution $\hat{\theta}_{\mathcal{M}}$, RMM stores p merged components with $2 \leq p \ll n$. These components define a low-dimensional subspace intended to capture the principal variation across task updates, enabling on-demand reconstruction of a target model. The integer p is the control parameter for the storage–accuracy trade-off: increasing p increases stored state and typically improves reconstruction fidelity.

For layer l , the relevant objects are the r -dimensional vectors contained in the low-rank delta factors A_i^l and B_i^l : rows of A_i^l and columns of B_i^l , where r is the compression rank. We define each row of A_i^l and each column of B_i^l as a *task vector*, and we call a fixed row or column index a *task vector position*. For a fixed position, the corresponding vector from model i is denoted $x_i \in \mathbb{R}^r$, and the collection $\{x_i\}_{i=1}^n$ contains the aligned vectors across all models. RMM is applied independently to every such position: it compresses the aligned task vectors by projecting them onto a shared basis and later reconstructs each vector from that basis. Each layer has $m + d$ positions, consisting of m rows from A_i^l and d columns from B_i^l .

For each fixed position, RMM estimates basis vectors $\{w_1, \dots, w_p\}$ with $w_j \in \mathbb{R}^r$ for $1 \leq j \leq p$. Their span defines the subspace $W = [w_1 \dots w_p] \in \mathbb{R}^{r \times p}$ onto which each task vector x_i is projected. The coefficient matrix $C \in \mathbb{R}^{n \times p}$ stores the reconstruction weights; its i -th row $[c_{i1}, \dots, c_{ip}]$ specifies the linear combination of basis vectors used to approximate x_i . The optimization problem is to choose W and C so that the aggregate squared reconstruction error over all n task vectors is minimized:

$$\min_{W, C} \sum_{i=1}^n \left\| x_i - \sum_{j=1}^p c_{ij} w_j \right\|_2^2. \quad (3)$$

This representation stores rp basis parameters and np coefficient parameters, for a total of $p(r + n)$ parameters per position instead of the rn parameters required to retain all task vectors explicitly. The parameter p therefore directly controls the performance–storage trade-off. Figure 3 depicts the construction for $p = 2$. In matrix notation, equation 3 becomes:

$$\min_{W, C} \|X - CW^T\|_F^2, \quad (4)$$

where $\mathbf{X} = [\mathbf{x}_1 \dots \mathbf{x}_n]^\top \in \mathbb{R}^{n \times r}$ stacks the task vectors at the fixed position. For a fixed $\mathbf{W}$, the coefficient optimum is the least-squares solution $\mathbf{C}^* = \mathbf{X}\mathbf{W}(\mathbf{W}^\top \mathbf{W})^{-1}$. Under the orthonormality constraint $\mathbf{W}^\top \mathbf{W} = \mathbf{I}_p$ with $p < r$, the objective reduces to:

$$\min_{\mathbf{W}} \|\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top\|_F^2 \quad \text{s.t.} \quad \mathbf{W}^\top \mathbf{W} = \mathbf{I}_p. \quad (5)$$

In this form, the stored quantities are $\mathbf{W} \in \mathbb{R}^{r \times p}$ and $\mathbf{X}\mathbf{W} \in \mathbb{R}^{n \times p}$, where $\mathbf{X}\mathbf{W}$ is the coefficient matrix $\mathbf{C}$ from equation 4.

Theorem 1 (Optimal Solution to equation 5). *Let $\mathbf{X} \in \mathbb{R}^{n \times r}$ be zero-centered and suppose $p < \min\{r, n\}$. An optimizer of equation 5 is obtained by taking the columns of $\mathbf{W}$ to be the top- p eigenvectors of the sample covariance matrix $\mathbf{X}^\top \mathbf{X}$. Equivalently, $\mathbf{W}^*$ is the matrix of the top- p right singular vectors of $\mathbf{X}$.*

Appendix A.1 gives the proof of Theorem 1. We now specify the two operational phases of RMM.

5.1 MERGING PHASE

The merging phase computes a compact shared-basis representation for the set of task-specific models. For each layer and each task vector position, defined as a fixed row in $\mathbf{A}_i^l$ or a fixed column in $\mathbf{B}_i^l$, we stack the corresponding vectors from all n models as $\mathbf{X} = [\mathbf{x}_1 \dots \mathbf{x}_n]^\top \in \mathbb{R}^{n \times r}$. We compute the empirical mean $\boldsymbol{\mu} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$ and center the matrix by setting $\mathbf{X} \leftarrow [\mathbf{x}_1 - \boldsymbol{\mu} \dots \mathbf{x}_n - \boldsymbol{\mu}]^\top$. We then apply SVD to the centered matrix, $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$, and select the first p right singular vectors as the optimal orthonormal basis for equation 5, namely $\mathbf{W}^* = \mathbf{V}_{[:,1:p]} \in \mathbb{R}^{r \times p}$. The coefficient matrix is $\mathbf{C}^* = \mathbf{X}\mathbf{W}^* \in \mathbb{R}^{n \times p}$, and its i -th row, $[c_{i1}, \dots, c_{ip}]$, stores the linear weights used to reconstruct $\mathbf{x}_i$.

After this offline phase, the original vectors $\{\mathbf{x}_i\}_{i=1}^n$ are no longer stored; each position retains only $\mathbf{W}^*$, $\mathbf{C}^*$, and $\boldsymbol{\mu}$. The per-position storage decreases from rn parameters for explicit retention of all task vectors to $p(r + n) + r$ parameters for the reversible representation. The full merging computation is performed before inference.

5.2 INFERENCE PHASE

At inference time, we follow the standard model-merging assumption that the input’s target task is either provided by the user or assigned by an oracle router to the appropriate model index. Given the selected task index i , RMM reconstructs the corresponding task vector $\mathbf{x}_i$ as $\hat{\mathbf{x}}_i$ from the stored tuple $(\mathbf{W}^*, \mathbf{C}^*, \boldsymbol{\mu})$ at each task vector position. The matrices $\mathbf{A}_i^l$ and $\mathbf{B}_i^l$ are then recovered by concatenating the reconstructed $\hat{\mathbf{x}}_i$ vectors over their respective row and column positions.

For each task vector position, $\mathbf{W}^* \in \mathbb{R}^{r \times p}$ contains the orthonormal basis, $\mathbf{C}^* \in \mathbb{R}^{n \times p}$ contains the task-specific coefficients, and $\boldsymbol{\mu} \in \mathbb{R}^r$ is the empirical mean of the aligned task vectors. Reconstruction uses the coefficient row for task i to form a linear combination of the basis directions and then adds back the mean:

$$\hat{\mathbf{x}}_i = \sum_{j=1}^p c_{ij}^* \mathbf{w}_j^* + \boldsymbol{\mu} = \mathbf{C}_{[i,:]}^* \mathbf{W}^{*\top} + \boldsymbol{\mu}.$$

The vector $\hat{\mathbf{x}}_i \in \mathbb{R}^r$ is the reconstructed approximation to the original task vector $\mathbf{x}_i$ that was discarded after merging. Applying this reconstruction independently to every row of $\mathbf{A}_i^l$, every column of $\mathbf{B}_i^l$, and every layer yields the reconstructed low-rank delta for task-specific model i , denoted $\hat{\Delta}_i$. Algorithm 1 summarizes both the offline merging phase and the inference-time reconstruction phase. We next evaluate RMM against standard merging baselines.

6 EXPERIMENTS

We evaluate RMM across multiple compression settings, task collections, and model architectures. The experiments measure whether the reversible representation preserves the performance of low-rank task-specific models while reducing aggregate storage, and they compare this behavior against conventional single-model merging baselines.

Algorithm 1 Reversible Model Merging (RMM)

```
1: Merging Phase:
2: Require: Low-rank deltas  $A_i^l$  and  $B_i^l$ , compression rank  $r$ , number of basis vectors  $p$ 
3: for  $l = 1$  to  $L$  do
4:   for each task vector position (each row in  $A_i^l$  or each column in  $B_i^l$ ) do
5:     Gather task vectors  $x_i$  of each model  $i$  ( $1 \leq i \leq n$ )
6:      $X \leftarrow [x_1 \dots x_n]^\top \in \mathbb{R}^{n \times r}$ 
7:      $\mu \leftarrow \sum_{i=1}^n x_i \in \mathbb{R}^r$ 
8:      $X \leftarrow [x_1 - \mu \dots x_n - \mu]^\top$ 
9:      $U, \Sigma, V^\top \leftarrow \text{SVD}(X)$ 
10:     $W^* \leftarrow V_{[:,1:p]} \in \mathbb{R}^{r \times p}$ 
11:     $C^* \leftarrow XW^* \in \mathbb{R}^{n \times p}$ 
12:    Store  $(W^*, C^*, \mu)$  and discard task vectors  $\{x_1 \dots x_n\}$ 
13:   end for
14: end for
15: Inference Phase:
16: Require: target task  $i$ , stored representation  $(W^*, C^*, \mu)$  for each task vector position
17: for  $l = 1$  to  $L$  do
18:   for each task vector position do
19:      $\hat{x}_i = C_{[i,:]}^* W^{*\top} + \mu$ 
20:   end for
21:    $\hat{A}_i^l = \text{Concatenate}(\hat{x}_i \text{ vectors across all row positions})$ 
22:    $\hat{B}_i^l = \text{Concatenate}(\hat{x}_i \text{ vectors across all column positions})$ 
23: end for
24:  $\hat{\Delta}_i = \{\hat{A}_i^l \hat{B}_i^l\}_{l=1}^L$ 
25: Inference with the reconstructed model  $\theta_{pre} + \hat{\Delta}_i$ 
```

6.1 EXPERIMENTAL SETUP

The primary evaluation uses eight RoBERTa-base models fine-tuned on eight tasks (Zhuang et al., 2021). Each model is compressed independently with either PT-SVD or LoRA at target ranks $r \in \{16, 32, 64, 128\}$. RMM is then applied to the resulting low-rank representations and compared with Task Arithmetic (TA) (Ilharco et al., 2023), TIES-merging (Yadav et al., 2023), and DARE (Yu et al., 2024). We report the performance of the unmerged compressed models as an upper-bound reference. Appendix A.2 provides additional evaluations on OPT-1.3b (Zhang et al., 2022) and ViT-B/32 (Radford et al., 2021), including settings with different numbers of tasks.

Evaluation follows the GLUE protocol (Wang et al., 2018), using the standard metric for each task. QNLI (question-answering inference), MRPC (paraphrase detection), SST-2 (sentiment classification), MNLI (multi-genre natural language inference), QQP (duplicate question detection), and RTE (textual entailment) are evaluated by accuracy. CoLA (linguistic acceptability) is evaluated by Matthews correlation, and STS-B (semantic textual similarity) is evaluated by Pearson correlation. The average over tasks is reported as the aggregate performance measure. We also report relative storage, defined as the fraction of parameters retained compared with storing all task-specific adapters. Baselines that store one merged model have relative cost $\frac{r}{rn} = \frac{1}{n}$. For RMM with p basis components, the relative storage cost is $\frac{p(r+n)+r}{rn}$, as derived in Section 5.1.

6.2 RESULTS AND DISCUSSION

Tables 1 and 2 report RMM and baseline performance under PT-SVD and LoRA compression, respectively. For both compression mechanisms and all target ranks $r \in \{16, 32, 64, 128\}$, RMM substantially exceeds TA, TIES, and DARE. At low ranks ($r = 16, 32$), the baselines collapse to average scores of only 31–42%, with especially poor performance on structurally sensitive tasks such as CoLA and STS-B. RMM with $p = 2$ provides large improvements, including 57.24% for PT-SVD and 51.41% for LoRA at $r = 16$, while $p = 3$ further increases the averages to 72.22% and 58.64% at the same rank. At higher ranks ($r = 64, 128$), where compression error is smaller,

r	Method	Storage	QNLI	MRPC	SST-2	MNLI	QQP	RTE	COLA	STS-B	Average
16	No merging	100%	92.02	85.68	93.69	85.21	87.19	77.98	58.05	90.59	83.80
	TA	13%	50.59	33.51	49.08	31.82	36.82	47.29	0.00	4.19	31.66
	TIES	13%	50.56	34.78	49.08	31.82	36.82	47.29	0.00	3.75	31.76
	DARE	13%	50.80	33.51	49.08	31.82	36.82	47.29	0.00	5.07	31.80
	RMM ($p = 2$)	50%	51.20	78.20	88.30	66.74	86.02	47.29	6.56	33.59	57.24
	RMM ($p = 3$)	69%	88.83	86.26	92.89	79.58	87.18	48.74	33.85	60.46	72.22
32	No merging	100%	92.31	85.86	94.15	86.38	88.62	78.34	59.33	90.68	84.46
	TA	13%	50.59	33.51	49.08	31.82	36.82	47.29	0.00	4.21	31.66
	TIES	13%	50.56	35.19	49.08	31.82	36.82	47.29	0.00	3.87	31.83
	DARE	13%	50.72	33.51	49.08	31.82	36.82	47.29	0.00	3.48	31.59
	RMM ($p = 2$)	44%	50.54	74.32	55.05	62.68	87.07	47.29	0.00	23.18	50.02
	RMM ($p = 3$)	59%	84.66	86.32	92.89	79.70	88.05	47.29	23.57	44.54	68.38
64	No merging	100%	92.51	86.03	94.38	87.18	90.15	78.70	61.11	90.72	85.10
	TA	13%	50.61	33.51	49.08	31.82	36.82	47.29	0.00	4.23	31.67
	TIES	13%	50.58	35.13	49.08	31.82	36.82	47.29	0.00	3.80	31.82
	DARE	13%	50.58	33.51	49.08	31.82	36.82	47.29	0.00	5.22	31.79
	RMM ($p = 2$)	41%	50.53	73.10	49.08	58.48	86.98	47.29	0.00	17.50	47.87
	RMM ($p = 3$)	55%	74.14	86.20	92.43	78.83	89.00	47.29	13.16	33.70	64.34
128	No merging	100%	92.68	86.14	94.50	87.33	91.05	79.78	60.16	90.74	85.30
	TA	13%	50.59	33.51	49.08	31.82	36.82	47.29	0.00	4.23	31.67
	TIES	13%	50.58	35.54	49.08	31.82	36.82	47.29	0.00	3.93	31.88
	DARE	13%	50.61	34.78	49.08	31.82	36.82	47.29	0.00	2.41	31.60
	RMM ($p = 2$)	39%	50.54	70.78	49.08	54.69	84.68	47.29	0.00	14.53	46.45
	RMM ($p = 3$)	52%	59.20	85.80	92.32	77.50	89.02	47.29	6.56	26.94	60.58

Table 1: GLUE performance for eight RoBERTa-base task models compressed with PT-SVD and merged at ranks $r = 16, 32, 64, 128$.

r	Method	Storage	QNLI	MRPC	SST-2	MNLI	QQP	RTE	COLA	STS-B	Average
16	No merging	100%	61.38	83.48	94.50	83.90	87.56	68.23	53.92	88.71	77.71
	TA	13%	50.59	33.51	72.13	34.79	64.46	47.29	0.00	21.81	40.57
	TIES	13%	50.63	33.68	74.31	35.64	67.45	47.29	0.00	26.38	41.92
	DARE	13%	50.63	33.51	73.74	35.97	65.35	47.29	0.00	21.62	41.01
	RMM ($p = 2$)	50%	51.13	34.14	90.71	60.48	84.41	47.29	0.00	43.14	51.41
	RMM ($p = 3$)	69%	55.92	38.14	93.12	82.95	87.30	47.29	6.56	57.81	58.64
32	No merging	100%	60.13	81.28	93.81	84.46	88.18	62.82	53.12	88.53	76.54
	TA	13%	50.91	68.06	54.70	33.77	65.50	55.96	0.00	-0.48	41.05
	TIES	13%	51.03	68.06	56.77	34.24	66.27	58.12	0.00	1.02	41.94
	DARE	13%	50.96	64.64	63.07	35.12	63.66	55.60	0.00	1.01	41.76
	RMM ($p = 2$)	44%	52.08	65.39	81.54	62.46	77.06	57.40	0.00	9.95	50.73
	RMM ($p = 3$)	59%	54.62	69.97	92.20	81.29	85.97	57.40	4.64	24.08	58.77
64	No merging	100%	59.62	84.12	94.15	85.20	88.75	67.15	58.31	88.70	78.25
	TA	13%	54.75	68.52	77.29	37.24	63.82	47.29	0.00	-3.53	39.80
	TIES	13%	54.82	68.75	80.73	37.67	63.82	47.29	0.00	-2.48	40.45
	DARE	13%	53.71	68.58	78.56	38.33	63.82	47.29	0.00	-4.28	39.88
	RMM ($p = 2$)	41%	55.74	69.28	87.27	42.81	77.72	47.29	0.00	4.91	48.13
	RMM ($p = 3$)	55%	59.07	70.14	90.83	79.11	82.06	47.29	0.00	14.70	55.40
128	No merging	100%	61.58	85.04	93.58	85.71	89.68	57.04	59.35	89.68	77.71
	TA	13%	51.49	69.22	54.47	37.16	63.27	47.29	0.00	-1.74	40.15
	TIES	13%	51.91	68.93	57.34	37.81	63.36	47.29	0.00	-0.25	40.80
	DARE	13%	51.97	68.99	58.60	35.64	63.33	47.29	0.00	-0.44	40.67
	RMM ($p = 2$)	39%	57.18	67.83	88.53	49.54	84.64	47.29	0.00	28.00	52.88
	RMM ($p = 3$)	52%	59.13	39.19	92.09	80.08	86.39	47.29	15.45	71.18	61.35

Table 2: GLUE performance for eight RoBERTa-base task models compressed with LoRA and merged at ranks $r = 16, 32, 64, 128$.

RMM remains the strongest method. With $p = 3$, it obtains 64.34% and 60.58% under PT-SVD, and 55.40% and 61.35% under LoRA, whereas the single-model baselines remain near 40%. Even the $p = 2$ variant retains a clear margin, indicating that reversible reconstruction mitigates the destructive interference observed in prior merging methods.

The relative advantage of RMM decreases as the compression rank increases. A plausible explanation is that higher-rank task representations occupy a larger and more expressive space, which can reduce alignment across tasks and make approximation by a shared low-dimensional basis more difficult. Nevertheless, RMM provides a consistent advantage across both compression regimes and

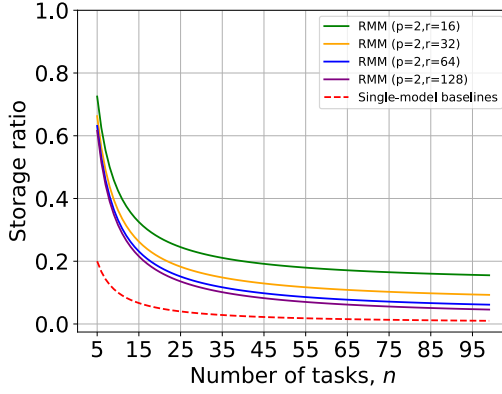


Figure 4: Storage scalability of RMM with $p = 2$. The storage ratio is the percentage of memory required relative to retaining all task-specific models.

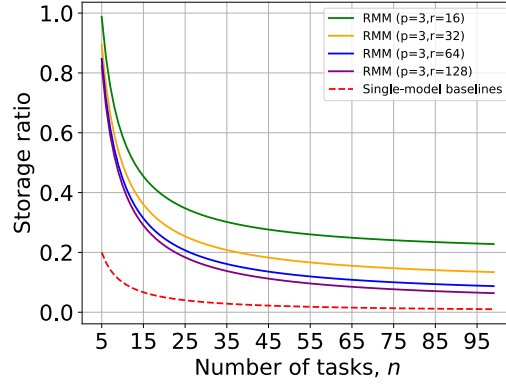


Figure 5: Storage scalability of RMM with $p = 3$. The storage ratio is the percentage of memory required relative to retaining all task-specific models.

all ranks. It preserves substantially higher task performance at the cost of additional stored components, yielding an explicit trade-off between accuracy and storage efficiency through the choice of p . This storage overhead is incurred in **offline memory**, typically CPU-resident and inexpensive, whereas task performance affects deployment quality directly; in this regime, modest CPU-side storage increases can be justified by substantially improved accuracy.

6.3 SCALABILITY OF RMM

We also analyze how RMM storage changes as the number of models n increases. Figures 4 and 5 plot the storage ratio, relative to storing all task-specific models, as a function of the number of tasks for $p = 2$ and $p = 3$. The curves show favorable scaling: as n grows, the relative storage cost decreases markedly. This behavior reflects the main storage property of reversible merging. Once the basis for a task vector position is fixed, an additional task requires only an additional row in the coefficient matrix, so storage grows sublinearly relative to explicit retention of all task-specific models. Conventional alternatives either store one merged model, which sacrifices task-specialized performance, or store every model separately, which grows linearly in n . These results support the use of RMM in large-scale multi-task, continual, and federated settings where hundreds of compressed models may need to be managed without discarding task-specific performance.

7 CONCLUSION

We introduced a reversible model-merging framework that specifies how multiple fine-tuned models can be stored, merged, and reconstructed under low-rank parameterization. Rather than constraining the merged representation to a single aggregate model, RMM stores a compact set of intermediate basis components, substantially fewer than the number of original models but more expressive than one merged update. This representation supports reconstruction of task-specific models while reducing total storage. The method exposes a controllable trade-off between reconstruction fidelity and storage efficiency, which is particularly relevant for low-rank compressed models. By representing fine-tuned updates through compact bases and task-specific coefficient mappings, RMM enables accurate deployment under explicit memory budgets.

Overall, RMM shifts model merging from a heuristic one-shot aggregation procedure to a structured reversible representation problem. This perspective enlarges the design space for efficient multi-task deployment and motivates future work on compression and reconstruction methods that explicitly optimize both storage cost and task-level performance.

REFERENCES

- Mircea Cimpoi, Subhransu Maji, Iasonas Kokkinos, Sammy Mohamed, and Andrea Vedaldi. Describing textures in the wild. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 3606–3613, 2014.
- Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In Jill Burstein, Christy Doran, and Thamar Solorio (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 2924–2936, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1300. URL <https://aclanthology.org/N19-1300/>.
- Antonio Andrea Gargiulo, Donato Crisostomi, Maria Sofia Bucarelli, Simone Scardapane, Fabrizio Silvestri, and Emanuele Rodola. Task singular vectors: Reducing task interference in model merging. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 18695–18705, 2025.
- Patrick Helber, Benjamin Bischke, Andreas Dengel, and Damian Borth. Eurosat: A novel dataset and deep learning benchmark for land use and land cover classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 12(7):2217–2226, 2019.
- Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. Cambridge University Press, 2 edition, 2012.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *CoRR*, abs/2106.09685, 2021. URL <https://arxiv.org/abs/2106.09685>.
- Chenyu Huang, Peng Ye, Tao Chen, Tong He, Xiangyu Yue, and Wanli Ouyang. Emr-merging: Tuning-free high-performance model merging. *Advances in Neural Information Processing Systems*, 37:122741–122769, 2024.
- Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=6t0Kwf8-jrj>.
- Xisen Jin, Xiang Ren, Daniel Preotiuc-Pietro, and Pengxiang Cheng. Dataless knowledge fusion by merging weights of language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=FCnohuR6AnM>.
- Jonathan Krause, Michael Stark, Jia Deng, and Li Fei-Fei. 3d object representations for fine-grained categorization. In *2013 IEEE International Conference on Computer Vision Workshops*, pp. 554–561, 2013. doi: 10.1109/ICCVW.2013.77.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- Michael S Matena and Colin A Raffel. Merging models with fisher-weighted averaging. *Advances in Neural Information Processing Systems*, 35:17703–17716, 2022.
- Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y. Ng. Reading digits in natural images with unsupervised feature learning. In *NIPS Workshop on Deep Learning and Unsupervised Feature Learning 2011*, 2011. URL http://ufldl.stanford.edu/housenumbers/nips2011_housenumbers.pdf.
- Bowen Ping, Shuo Wang, Hanqing Wang, Xu Han, Yuzhuang Xu, Yukun Yan, Yun Chen, Baobao Chang, Zhiyuan Liu, and Maosong Sun. Delta-come: Training-free delta-compression with mixed-precision for large language models. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 31056–31077. Curran Associates, Inc., 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/37664246a1e07e212ddacea6e5a523f2-Paper-Conference.pdf.

- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In Marina Meila and Tong Zhang (eds.), *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 8748–8763. PMLR, 18–24 Jul 2021. URL <https://proceedings.mlr.press/v139/radford21a.html>.
- Simo Ryu, Seunghyun Seo, and Jaeyun Yoo. Efficient storage of fine-tuned models via low-rank approximation of weight residuals. *arXiv preprint arXiv:2305.18425*, 2023.
- Johannes Stallkamp, Marc Schlipsing, Jan Salmen, and Christian Igel. The german traffic sign recognition benchmark: A multi-class classification competition. In *The 2011 International Joint Conference on Neural Networks*, pp. 1453–1460, 2011. doi: 10.1109/IJCNN.2011.6033395.
- Dennis Tang, Prateek Yadav, Yi-Lin Sung, Jaehong Yoon, and Mohit Bansal. LoRA merging with SVD: Understanding interference and preserving performance. In *ICML 2025 Workshop on Reliable and Responsible Foundation Models*, 2025. URL <https://openreview.net/forum?id=t9FrMviTaP>.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In Tal Linzen, Grzegorz Chrupała, and Afra Alishahi (eds.), *Proceedings of the 2018 EMNLP Workshop Black-boxNLP: Analyzing and Interpreting Neural Networks for NLP*, pp. 353–355, Brussels, Belgium, November 2018. Association for Computational Linguistics. doi: 10.18653/v1/W18-5446. URL <https://aclanthology.org/W18-5446/>.
- Xin Wang, Samiul Alam, Zhongwei Wan, Hui Shen, and Mi Zhang. SVD-LLM v2: Optimizing singular value truncation for large language model compression. In Luis Chiruzzo, Alan Ritter, and Lu Wang (eds.), *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 4287–4296, Albuquerque, New Mexico, April 2025a. Association for Computational Linguistics. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.217. URL <https://aclanthology.org/2025.naacl-long.217/>.
- Xin Wang, Yu Zheng, Zhongwei Wan, and Mi Zhang. SVD-LLM: Truncation-aware singular value decomposition for large language model compression. In *The Thirteenth International Conference on Learning Representations*, 2025b. URL <https://openreview.net/forum?id=LNyIUouhdt>.
- Yongxian Wei, Anke Tang, Li Shen, Zixuan Hu, Chun Yuan, and Xiaochun Cao. Modeling multi-task model merging as adaptive projective gradient descent. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=EgqKRSR5Pa>.
- Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International conference on machine learning*, pp. 23965–23998. PMLR, 2022.
- Jianxiong Xiao, James Hays, Krista A. Ehinger, Aude Oliva, and Antonio Torralba. Sun database: Large-scale scene recognition from abbey to zoo. In *2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pp. 3485–3492, 2010. doi: 10.1109/CVPR.2010.5539970.
- Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-merging: Resolving interference when merging models. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=xtaX3WycJl>.
- Enneng Yang, Zhenyi Wang, Li Shen, Shiwei Liu, Guibing Guo, Xingwei Wang, and Dacheng Tao. Adamerging: Adaptive model merging for multi-task learning. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=nZP6NgD3QY>.

- Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *Forty-first International Conference on Machine Learning*, 2024.
- Juzheng Zhang, Jiacheng You, Ashwinee Panda, and Tom Goldstein. LoRI: Reducing cross-task interference in multi-task low-rank adaptation. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=b8cW86QcOD>.
- Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.
- Ziyu Zhao, Tao Shen, Didi Zhu, Zexi Li, Jing Su, Xuwu Wang, and Fei Wu. Merging loRAs like playing LEGO: Pushing the modularity of loRA to extremes through rank-wise clustering. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=j6fsbpAl1N>.
- Shenghe Zheng, Hongzhi Wang, Chenyu Huang, Xiaohui Wang, Tao Chen, Jiayuan Fan, Shuyue Hu, and Peng Ye. Decouple and orthogonalize: A data-free framework for lora merging. *arXiv preprint arXiv:2505.15875*, 2025.
- Liu Zhuang, Lin Wayne, Shi Ya, and Zhao Jun. A robustly optimized BERT pre-training approach with post-training. In Sheng Li, Maosong Sun, Yang Liu, Hua Wu, Kang Liu, Wanxiang Che, Shizhu He, and Gaoqi Rao (eds.), *Proceedings of the 20th Chinese National Conference on Computational Linguistics*, pp. 1218–1227, Huhhot, China, August 2021. Chinese Information Processing Society of China. URL <https://aclanthology.org/2021.ccl-1.108/>.

A APPENDIX

A.1 PROOF OF THEOREM 1

Let $\mathbf{A} = \mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top$. By the Frobenius norm identity, $\|\mathbf{A}\|_F^2 = \text{Tr}(\mathbf{A}^\top \mathbf{A})$, where $\text{Tr}(\cdot)$ denotes the trace of a square matrix. Substituting the definition of $\mathbf{A}$ yields:

$$\|\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top\|_F^2 = \text{Tr}((\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top)^\top (\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top)).$$

The matrix product inside the trace expands as:

$$\begin{aligned} (\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top)^\top (\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top) &= (\mathbf{X}^\top - \mathbf{W}\mathbf{W}^\top \mathbf{X}^\top)(\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top) \\ &= \mathbf{X}^\top \mathbf{X} - \mathbf{X}^\top \mathbf{X}\mathbf{W}\mathbf{W}^\top - \mathbf{W}\mathbf{W}^\top \mathbf{X}^\top \mathbf{X} \\ &\quad + \mathbf{W}\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}\mathbf{W}^\top. \end{aligned}$$

Taking the trace of this expression and using cyclicity of trace together with $\mathbf{W}^\top \mathbf{W} = \mathbf{I}_p$ gives:

$$\begin{aligned} \|\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top\|_F^2 &= \text{Tr}(\mathbf{X}^\top \mathbf{X}) - 2\text{Tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}) + \text{Tr}(\mathbf{W}^\top \mathbf{W}\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}) \\ &= \text{Tr}(\mathbf{X}^\top \mathbf{X}) - 2\text{Tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}) + \text{Tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}) \\ &= \text{Tr}(\mathbf{X}^\top \mathbf{X}) - \text{Tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}). \end{aligned}$$

Because $\text{Tr}(\mathbf{X}^\top \mathbf{X})$ does not depend on $\mathbf{W}$, minimizing $\|\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top\|_F^2$ is equivalent to maximizing $\text{Tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W})$. Therefore,

$$\min_{\mathbf{W}^\top \mathbf{W} = \mathbf{I}_p} \|\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top\|_F^2 \iff \max_{\mathbf{W}^\top \mathbf{W} = \mathbf{I}_p} \text{Tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}).$$

The Rayleigh quotient theorem (Horn & Johnson, 2012) implies that the maximizer is obtained by setting the columns of $\mathbf{W}$ to the top- p eigenvectors of $\mathbf{X}^\top \mathbf{X}$. This is exactly the principal component analysis (PCA) solution: the selected p -dimensional subspace maximizes projected variance and equivalently minimizes squared reconstruction error. If $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ is the SVD of $\mathbf{X}$, then $\mathbf{X}^\top \mathbf{X} = \mathbf{V}\mathbf{\Sigma}^2\mathbf{V}^\top$, so the eigenvectors of $\mathbf{X}^\top \mathbf{X}$ are the right singular vectors, i.e., the columns of $\mathbf{V}$. The PCA solution can therefore be computed by selecting the top- p right singular vectors of $\mathbf{X}$. In implementation, this SVD-based computation is typically more numerically stable and computationally preferable to explicitly forming $\mathbf{X}^\top \mathbf{X}$, especially for large r .

r	Method	Storage	QNLI	MRPC	SST-2	MNLI	QQP	BOOLQ	Average
16	No merging	100%	91.74	71.54	95.30	84.19	88.14	71.47	83.73
	Task-Arithmetic	17%	50.34	37.16	58.49	34.43	63.30	41.87	47.60
	TIES	17%	49.79	36.75	61.24	35.57	63.55	40.67	47.93
	DARE	17%	49.95	37.68	56.08	34.51	63.39	42.78	47.40
	RMM ($p = 2$)	63%	88.12	44.00	93.58	73.06	85.63	59.69	74.01
	RMM ($p = 3$)	85%	91.58	49.04	95.64	80.31	87.52	65.50	78.27
32	No merging	100%	92.06	72.17	95.41	84.60	89.17	72.45	84.31
	Task-Arithmetic	17%	50.28	37.22	58.94	34.57	63.33	41.93	47.71
	TIES	17%	49.77	36.93	61.93	35.69	63.65	40.73	48.12
	DARE	17%	50.39	40.87	56.19	34.96	63.59	43.33	48.22
	RMM ($p = 2$)	56%	86.66	41.91	91.74	71.75	85.68	56.39	72.36
	RMM ($p = 3$)	76%	91.85	44.81	94.84	80.21	88.43	63.52	77.28
64	No merging	100%	92.26	72.12	95.41	84.64	90.07	72.84	84.56
	Task-Arithmetic	17%	50.28	37.28	59.06	34.56	63.34	42.08	47.77
	TIES	17%	49.75	37.04	61.81	35.84	63.74	40.76	48.16
	DARE	17%	50.16	40.52	58.83	35.59	63.35	42.54	48.50
	RMM ($p = 2$)	53%	83.87	42.72	89.45	69.76	85.70	53.43	70.82
	RMM ($p = 3$)	71%	92.07	41.97	94.50	79.70	88.73	61.96	76.49
128	No merging	100%	92.35	72.41	95.41	84.67	90.93	73.09	84.81
	Task-Arithmetic	17%	50.32	37.39	58.94	34.63	63.37	42.08	47.79
	TIES	17%	49.79	37.04	62.04	35.99	63.74	40.55	48.19
	DARE	17%	50.03	38.14	57.91	34.72	63.31	42.11	47.70
	RMM ($p = 2$)	52%	81.38	42.78	84.75	67.66	83.22	51.99	68.63
	RMM ($p = 3$)	69%	92.28	41.80	93.69	79.42	88.76	59.69	75.94

Table 3: GLUE and related benchmark performance for six OPT-1.3b task models compressed with PT-SVD and merged at ranks $r = 16, 32, 64, 128$.

r	Method	Storage	MNIST	GTSRB	SVHN	Cars	EuroSAT	DTD	SUN397	Average
16	No merging	100%	99.61	97.22	97.06	69.20	99.67	75.21	72.72	87.24
	Task-Arithmetic	14%	47.19	28.29	38.64	56.27	57.52	41.38	62.92	47.46
	TIES	14%	60.14	34.52	49.03	58.55	61.70	43.67	62.87	52.93
	DARE	14%	53.80	31.00	44.96	57.60	56.93	43.99	62.42	50.10
	RMM ($p = 2$)	55%	99.16	88.52	94.89	66.61	97.44	60.64	67.86	82.16
	RMM ($p = 3$)	76%	99.58	95.65	96.54	68.30	98.93	67.18	69.50	85.10
32	No merging	100%	99.65	98.45	97.35	73.04	99.78	78.72	75.52	88.93
	Task-Arithmetic	14%	47.90	28.31	38.68	56.24	57.52	41.38	63.00	47.58
	TIES	14%	61.25	35.28	49.95	58.80	62.41	44.04	63.05	53.54
	DARE	14%	53.05	31.04	46.34	58.54	57.74	42.87	62.87	50.35
	RMM ($p = 2$)	49%	98.98	88.23	94.29	68.32	96.81	61.91	68.36	82.41
	RMM ($p = 3$)	67%	99.50	96.53	96.68	70.90	98.93	70.43	71.23	86.31
64	No merging	100%	99.67	98.68	97.41	75.76	99.85	79.15	77.73	89.75
	Task-Arithmetic	14%	47.99	28.30	38.71	56.21	57.59	41.38	63.22	47.63
	TIES	14%	61.83	35.93	50.28	59.08	63.07	44.15	63.17	53.93
	DARE	14%	54.78	32.79	45.05	58.23	61.30	42.87	62.77	51.11
	RMM ($p = 2$)	46%	98.46	84.43	92.83	69.36	95.59	62.39	69.77	81.83
	RMM ($p = 3$)	62%	99.45	96.14	96.33	72.84	98.56	72.07	73.85	87.03
128	No merging	100%	99.68	98.75	97.46	76.99	99.85	79.04	78.59	90.05
	Task-Arithmetic	14%	48.00	28.31	38.73	56.21	57.63	41.44	63.35	47.67
	TIES	14%	62.08	36.36	50.48	59.23	63.41	44.68	63.25	54.21
	DARE	14%	55.16	32.07	45.47	58.55	61.85	43.09	62.85	51.29
	RMM ($p = 2$)	44%	97.22	78.56	90.63	70.22	94.00	61.49	71.03	80.45
	RMM ($p = 3$)	59%	99.28	94.59	95.95	74.24	98.19	71.86	76.02	87.16

Table 4: Vision classification performance for seven ViT-B/32 task models compressed with PT-SVD and merged at ranks $r = 16, 32, 64, 128$.

A.2 ADDITIONAL RESULTS

This section reports extended evaluations for two additional pre-trained backbones, OPT-1.3b and ViT-B/32. For **OPT-1.3b**, evaluation uses six natural language understanding tasks from GLUE and related benchmarks: QNLI, MRPC, SST-2, MNLI, QQP, and BoolQ (Clark et al., 2019). For **ViT-B/32**, evaluation uses seven vision classification datasets: MNIST (LeCun et al., 1998), GTSRB (Stallkamp et al., 2011), SVHN (Netzer et al., 2011), Cars (Krause et al., 2013), EuroSAT (Helber et al., 2019), DTD (Cimpoi et al., 2014), and SUN397 (Xiao et al., 2010).

Table 3 reports results for six OPT-1.3b models compressed with PT-SVD. Both RMM configurations, $p = 2$ and $p = 3$, consistently outperform the baselines; the $p = 3$ configuration further reduces the gap to the individual compressed models while storing substantially fewer parameters. Table 4 reports the corresponding evaluation for seven ViT-B/32 models. On the vision tasks, the effect is especially strong: with $p = 3$, RMM obtains average accuracy within 3% of the original models while requiring only 59% of the storage needed to retain every model at $r = 128$. These results indicate that RMM reconstructs compressed task models with high fidelity while providing meaningful storage reduction.

A.3 RESULTS FOR 5 TASKS

The motivating experiments in Section 4 use RoBERTa-base as the pretrained backbone and evaluate merging baselines on five GLUE tasks: QNLI, MRPC, SST-2, MNLI, and QQP. Figures 1 and 2 in Section 4 summarize the corresponding performance trends. For completeness, Table 5 provides the LoRA results associated with Figure 1, including the RMM improvements, and Table 6 provides the PT-SVD results associated with Figure 2.

r	Method	Storage	QNLI	MRPC	SST-2	MNLI	QQP	Average
16	No merging	100%	61.38	83.48	94.50	83.90	87.56	82.16
	Task-Arithmetic	20%	50.61	33.51	73.85	34.23	64.58	51.36
	TIES	20%	50.65	33.51	75.57	35.83	66.97	52.51
	DARE	20%	50.61	33.51	79.36	34.70	63.40	52.32
	RMM ($p = 2$)	73%	59.64	68.64	93.00	82.68	86.89	78.17
32	No merging	100%	60.13	81.28	93.81	84.46	88.18	81.57
	Task-Arithmetic	20%	51.18	67.30	58.83	34.33	68.56	56.04
	TIES	20%	51.36	68.17	59.52	34.76	71.46	57.05
	DARE	20%	50.83	64.87	68.12	35.54	68.93	57.66
	RMM ($p = 2$)	66%	57.72	67.54	92.55	81.80	86.13	77.15
64	No merging	100%	59.62	84.12	94.15	85.20	88.75	82.37
	Task-Arithmetic	20%	54.74	68.93	80.28	35.70	36.82	55.29
	TIES	20%	55.26	68.93	78.78	36.76	36.82	55.31
	DARE	20%	54.71	68.81	74.54	38.72	36.82	54.72
	RMM ($p = 2$)	63%	59.82	67.48	91.51	83.21	84.41	77.29
128	No merging	100%	61.58	85.04	93.58	85.71	89.68	83.12
	Task-Arithmetic	20%	52.52	68.70	54.82	37.83	63.46	55.47
	TIES	20%	52.55	69.04	57.80	38.91	63.72	56.40
	DARE	20%	55.04	69.62	54.47	38.92	64.12	56.43
	RMM ($p = 2$)	62%	60.21	34.96	91.86	78.59	86.65	70.45

Table 5: GLUE performance for five RoBERTa-base task models compressed with LoRA and merged at ranks $r = 16, 32, 64, 128$.

r	Method	Storage	QNLI	MRPC	SST-2	MNLI	QQP	Average
16	No merging	100%	92.02	85.68	93.69	85.21	87.19	88.76
	Task-Arithmetic	20%	53.03	35.19	49.08	31.82	36.82	41.19
	TIES	20%	55.46	54.32	49.08	31.82	37.21	45.58
	DARE	20%	53.69	58.67	49.08	31.82	37.08	46.07
	RMM ($p = 2$)	73%	79.72	85.16	90.83	76.58	86.09	83.68
32	No merging	100%	92.31	85.86	94.15	86.38	88.62	89.46
	Task-Arithmetic	20%	53.23	35.59	49.08	31.82	36.82	41.31
	TIES	20%	55.78	55.19	49.08	31.82	37.62	45.90
	DARE	20%	55.89	33.74	49.20	31.82	36.85	41.50
	RMM ($p = 2$)	66%	63.87	82.78	91.28	77.12	87.05	80.42
64	No merging	100%	92.51	86.03	94.38	87.18	90.15	90.05
	Task-Arithmetic	20%	53.49	35.88	49.08	31.82	36.82	41.42
	TIES	20%	56.14	55.94	49.08	31.82	38.08	46.21
	DARE	20%	53.51	33.68	49.08	31.82	36.82	40.98
	RMM ($p = 2$)	63%	56.12	81.22	91.06	76.74	88.07	78.64
128	No merging	100%	92.68	86.14	94.50	87.33	91.05	90.34
	Task-Arithmetic	20%	53.60	36.06	49.08	31.82	36.82	41.48
	TIES	20%	56.40	56.35	49.08	31.82	38.61	46.45
	DARE	20%	50.52	34.96	49.08	31.82	36.92	40.66
	RMM ($p = 2$)	62%	54.11	79.19	90.60	74.49	87.46	77.17

Table 6: GLUE performance for five RoBERTa-base task models compressed with PT-SVD and merged at ranks $r = 16, 32, 64, 128$.